CO3404 Distributed Systems

Complete Revision Guide 2025/26

START HERE

How to Use This Guide

Exam Format & Strategy

Topic Priority Map

Lecturer Practice Order

Past Paper Question Bank

Starter Glossary

WEEK 1 — HIGHEST PRIORITY

1. Cryptography Foundations W1

2. TLS, HTTPS & Certificates W1

3. IAM: OAuth, OIDC, JWT, SSO W1

4. Monolith vs Microservices W1

WEEK 2 — HIGH PRIORITY

5. APIs: REST, Endpoints, JSON W2

6. Express, Middleware & CORS W2

7. API Documentation & OpenAPI W2

8. Messaging: Queues, Pub-Sub, Events W2

9. Cloud: IaaS / PaaS / SaaS W2

WEEK 3 — MEDIUM PRIORITY

10. Resilience & Elastic Compute W3

11. Containers, Docker & Compose W3

12. Kubernetes & Rolling Updates W3

13. Data Layer, ACID & SAGA W3

14. Networking, VNets & CIDR W3

15. Infrastructure as Code: Terraform W3

EXAM PREPARATION

16. Code-Reading Practice

17. Key Diagrams to Memorise

18. Exam Technique

19. Rapid-Fire Review

CO3404 Distributed Systems

Complete Revision Guide — Built from lecture slides and 6 past papers (2022/23–2024/25)

How to Use This Guide

IF YOU OPEN THIS ON YOUR PHONE AND DO NOT KNOW WHERE TO START

1. Read **Starter Glossary**.
2. Study one high-priority topic from Week 1.
3. Go to **Past Paper Question Bank** and pick one recommended question.
4. Answer it under time pressure.
5. Open the dropdown answer-focus and check what you missed.

This guide is built for someone starting from zero. Every topic is explained from scratch before any technical language is used. Do not try to read it cover to cover in one sitting. Instead, use active revision:

1. **Read** one section slowly until the idea makes sense in plain English.
2. **Close** the guide and try to recall the key points from memory.
3. **Attempt** the exam-style question without looking at the model answer.
4. **Compare** your answer to the model answer and note the gaps.
5. **Return** to weak sections 2–3 days later (spaced repetition).

HIGHEST-VALUE HABIT

After reading a section, close this guide and write a one-page summary from memory. The act of retrieving information is what makes it stick. Passive re-reading does very little.

Past Paper Question Bank

Every question the lecturer recommended for revision is listed below with a **full model answer**. Use the workflow: read the question, set a timer (1 mark = 1 minute), write your answer from memory, then open the dropdown to compare.

HOW TO USE THESE ANSWERS

Do NOT read the model answer first. Write your own attempt under timed conditions, then compare. Note every point you missed. The gap list IS your revision target.

2022/23 Main Paper

Q2B — QUEUE VS PUB-SUB (15 MARKS, ~18 MINS)

Compare and contrast the shared message queue pattern with the publisher-subscriber pattern. With a diagram, give a microservice use-case for each and explain why you recommend that pattern.

▶ ▶ Show full model answer

Q6B — VM VS CONTAINER (10 MARKS, ~12 MINS)

With a diagram, explain the structure, benefits and key differences between a VM and a container, with an example of when to use each.

▶ ▶ Show full model answer

2022/23 Reassessment Paper

Q1A — MONOLITH VS MICROSERVICES (15 MARKS, ~18 MINS)

Compare and contrast a monolithic system with a distributed microservice-based system from technical and business standpoints. Five key points, up to 3 marks each.

▶ ▶ Show full model answer

Q2A — HATEOAS + RICHARDSON'S MATURITY MODEL (13 MARKS, ~16 MINS)

Do all RESTful APIs conform to HATEOAS? Explain HATEOAS, give an example, an advantage, a disadvantage, and relate to Richardson's Maturity Model.

▶ ▶ Show full model answer

Q2B — THREE OPENAPI DOCUMENTATION APPROACHES (12 MARKS, ~14 MINS)

Using the OpenAPI standard, describe design-first, API-first and code-first approaches. Justify where each is useful.

▶ ▶ Show full model answer

Q4 — FULL HTTPS/TLS WITH SIX CRYPTOGRAPHIC TECHNIQUES (25 MARKS, ~30 MINS)

With a diagram, explain how HTTPS works from web request to secure session. Show where Symmetric Encryption, Asymmetric Encryption, HMAC, Digital Signature, Digital Certificate and Key Exchange are used.

▶ ▶ Show full model answer

Q6 — 12 IAM TERMS (25 MARKS, ~30 MINS)

Define each IAM term and provide the context in which it is used.

▶ ▶ Show full model answer

2023/24 Main Paper

Q1A — EXPRESS MIDDLEWARE (7 MARKS, ~8 MINS)

With the aid of an example, explain the concept and operation of NodeJS Express middleware.

▶ ▶ Show full model answer

Q2B — ALICE SENDS A SECURE SIGNED MESSAGE TO BOB (7 MARKS, ~8 MINS)

Explain the asymmetric encryption process of Alice securely sending a message to Bob such that only Bob can read it and Bob knows it came from Alice.

▶ ▶ Show full model answer

Q3A — MICROSERVICE RATIONALE AND PROPERTIES (6 MARKS, ~7 MINS)

Explain the rationale and key properties of a microservice.

▶ ▶ Show full model answer

Q4 — ALL 8 KUBERNETES COMPONENTS (25 MARKS, ~30 MINS)

Explain the role of each K8s component: Control Plane, Kubelet, Node, Pod, ReplicaSet, Deployment, ConfigMap, Service (with ClusterIP/NodePort/LoadBalancer).

▶ ▶ Show full model answer

2023/24 Refer/Defer Paper

Q2A — THREE AVAILABILITY ZONE RESILIENCE BENEFITS (6 MARKS, ~7 MINS)

Provide three clear benefits an availability zone offers regarding system resilience. Use the format: "An AZ offers [benefit] which offers protection of [X] by [mechanism]."

▶ ▶ Show full model answer

Q3A — FOUR MONOLITH/MICROSERVICE ADVANTAGES AND DISADVANTAGES (8 MARKS, ~10 MINS)

Provide four advantages and/or disadvantages of monolithic and microservice architectures. 2 marks per example.

▶ ▶ Show full model answer

Q3B — THICK VS THIN EVENTS (6 MARKS, ~7 MINS)

Explain thick and thin events in event-driven architecture and when you would choose one over the other.

▶ ▶ Show full model answer

Q3D — TLS SESSION KEYS (4 MARKS, ~5 MINS)

Explain session keys in TLS, state how many there are and what each is used for.

▶ ▶ Show full model answer

Q4B — TLS HANDSHAKE STEP BY STEP (15 MARKS, ~18 MINS)

Explain each step from TCP connection to bi-directional encrypted data in transit. No need to explain key generation algorithms — just refer to their use.

▶ ▶ Show full model answer

Q5A — KUBERNETES ROLLING UPGRADES (7 MARKS, ~8 MINS)

Explain what rolling upgrades mean in Kubernetes and how they work.

▶ ▶ Show full model answer

Q5B — DOCKER COMPOSE YAML LINE-BY-LINE (12 MARKS, ~14 MINS)

Explain what the declarations at specific lines of a docker compose.yaml file achieve.

▶ ▶ Show full model answer

Q6C — SHARING DATA ACROSS MICROSERVICES: SYNC VS ASYNC (7 MARKS, ~8 MINS)

Discuss synchronous and the two asynchronous approaches to data sharing across microservices, with advantages and disadvantages.

▶ ▶ Show full model answer

2024/25 Main Paper

Q1A — API ENDPOINT DESIGN: BOOKSTORE (6 MARKS, ~7 MINS)

Design an API endpoint to retrieve a book by media type and genre with optional filters for author, title and max price. Explain the design and give an example call.

▶ ▶ Show full model answer

Q3B — KUBERNETES VS DOCKER COMPOSE (6 MARKS, ~7 MINS)

Justify two advantages Kubernetes offers over Docker Compose in a distributed enterprise system.

▶ ▶ Show full model answer

Q3C — ROLLING UPDATES WITH DIAGRAM (13 MARKS, ~16 MINS)

With a diagram, explain how Kubernetes enables rolling updates with no downtime using Deployment and ReplicaSet.

▶ ▶ Show full model answer

Q4B — MESSAGING PATTERNS: HTTP, QUEUE, EVENT (15 MARKS, ~18 MINS)

For each of HTTP, message queue and event messaging: one advantage, one disadvantage, one use-case. Up to 5 marks per pattern.

▶ ▶ Show full model answer

2024/25 Resit Paper

Q1A — HTTPS PROPERTIES OVER HTTP (8 MARKS, ~10 MINS)

Explain the key properties HTTPS offers over HTTP and when you might choose one over the other.

▶ ▶ Show full model answer

Q1B — WHY DOCUMENT APIS + OPENAPI ELEMENTS (8 MARKS, ~10 MINS)

Why is documenting API endpoints important? What key elements are included using the OpenAPI standard?

▶ ▶ Show full model answer

Q1C — EXPRESS MIDDLEWARE PURPOSE AND USE-CASE (9 MARKS, ~11 MINS)

Explain the purpose and benefits of Express middleware. Give a use-case with a middleware function and explain how and when it is called.

▶ ▶ Show full model answer

Q2A — IAAS VS PAAS, WHEN TO CHOOSE (5 MARKS, ~6 MINS)

Explain the difference between IaaS and PaaS. When would you choose one over the other?

▶ ▶ Show full model answer

Q2B — RESILIENCE + THREE EXAMPLES (10 MARKS, ~12 MINS)

Explain "resilience" in relation to cloud-based systems and provide three examples of how to make solutions resilient.

▶ ▶ Show full model answer

Q2C — LOAD BALANCER WITH DIAGRAM (10 MARKS, ~12 MINS)

With a diagram, explain the purpose and operation of a load balancer in elastic-compute solutions.

▶ ▶ Show full model answer

Q3B — MESSAGE QUEUE FOR LOOSE COUPLING (8 MARKS, ~10 MINS)

Explain how a message queue loosely couples microservices and increases resilience. Provide a use-case.

▶ ▶ Show full model answer

Q3C — PUBLISHER-SUBSCRIBER PATTERN WITH USE-CASE (8 MARKS, ~10 MINS)

Explain the publisher-subscriber messaging pattern and apply it to a relevant use-case.

▶ ▶ Show full model answer

Q4C — FIVE BUSINESS BENEFITS OF MICROSERVICES OVER MONOLITH (10 MARKS, ~12 MINS)

Provide and justify five business benefits of a distributed microservice architecture over a monolith.

▶ ▶ Show full model answer

Q5C — JSON STRUCTURE: BOB, AGE 25, THREE CHILDREN (5 MARKS, ~5 MINS)

Create an appropriate JSON structure for a person aged 25 named Bob Martin with children Jim, Jane and Susan.

▶ ▶ Show full model answer

FASTEST HIGH-VALUE PRACTICE SET

If you are short on time, do these five first (they cover the most marks and most-tested topics):
2022/23 resit Q4 (TLS/HTTPS 25 marks) → **2022/23 resit Q6** (IAM terms 25 marks) → **2023/24 main Q4** (K8s components 25 marks) → **2024/25 main Q4b** (messaging patterns 15 marks) → **2023/24 refer/defer Q4b** (TLS handshake 15 marks).

Exam Format & Strategy

Year	Questions available	Answer	Time	Marks per question
2022/23	6	4	2 hrs	25 each = 100
2023/24 onwards	5–6	3	1.5 hrs	25 each = 75 marks = 100%

From 2023/24 the format is **3 questions × 25 marks in 90 minutes**. That is 30 minutes per question. You have 5–6 questions to choose from, so you have genuine choice — skip the one you know least and pick the three you know best.

TIME BUDGET PER 25-MARK QUESTION

- 5-mark part: ~6 minutes — definition + one example is usually enough
- 10-mark part: ~12 minutes — clear explanation + diagram + example

- 15-mark part: ~18 minutes — structured answer with labelled diagram, multiple points, justification

DO NOT WRITE A WALL OF TEXT

Use numbered steps for processes (TLS, OIDC). Use bullet points for lists of advantages/disadvantages. Draw diagrams wherever the question says "with the aid of a diagram" — a good diagram with labels can earn 3–5 marks on its own.

Topic Priority Map

Based on analysis of all 6 past papers (2022/23 main, 2022/23 resit, 2023/24 main, 2023/24 refer/defer, 2024/25 main, 2024/25 resit):

LECTURER STRATEGY NOTE NEW

The lecturer's *Revision strategy* sheet reinforces the same pattern as the papers, and also flags several **new-this-year topics** that deserve explicit attention: **Bastion host**, **Event Notification**, **Event Carried State Transfer (ECST)**, **Network peering**, **3-tier architecture**, and **SQL injection**. These are covered in this guide even where older papers do not test them heavily.

Topic	Papers appeared in
Security: TLS handshake, digital signatures, certificates	6 / 6
Architecture: monolith vs microservices	6 / 6
Messaging: queues, pub-sub, event-driven	6 / 6
IAM: OAuth, OIDC flow, JWT, SSO	5 / 6
Cloud: IaaS/PaaS/SaaS, shared responsibility	5 / 6
Containers / Kubernetes	5 / 6
APIs: REST, endpoints, express, CORS	5 / 6
Resilience, availability zones, load balancers	4 / 6

Data: ACID, SAGA, read replicas, sharding	4 / 6
Terraform / IaC	2 / 6 (but 20 marks in most recent exam)
Networking / CIDR / VNets	2 / 6 (but full 25-mark Q in 2022/23)
API documentation, OpenAPI	3 / 6

Lecturer Practice Order

This section turns the lecturer's revision strategy into a practical question order. If you want the fastest route to exam confidence, use this list for your timed practice rather than choosing questions randomly.

> **HOW TO USE THIS LIST**
>
> 1. Read the matching guide section first.
> 2. Set a timer using roughly **1 mark per minute**.
> 3. Write the answer without notes.
> 4. Mark yourself against the question wording and the guide.
> 5. Redo weak answers 2-3 days later.

Best order to build confidence

Phase	Focus	Recommended past-paper questions
Phase 1	Highest-value repeated topics	2022/23 main: Q2b, Q6b. 2022/23 resit: Q1a, Q4, Q6. 20
Phase 2	Security and architecture depth	Certificate / TLS / signature questions, API gateway ques
Phase 3	New-this-year topics	Bastion host, Event Notification, ECST, network peering,
Phase 4	Final timed mixed practice	Pick any three 25-mark questions you would choose in th

Questions explicitly highlighted by the lecturer

- **2022/23 main:** Q2b, Q6b
- **2022/23 resit:** Q1a, Q2b, Q4, Q6
- **2023/24 main:** Q1a, Q2b, Q3a, Q4

- **2023/24 refer/defer:** Q2a, Q3a, Q3b, Q3d, Q4b, Q5a, Q5b, Q6c
- **2024/25 main:** Q1a, Q3b, Q3c, Q4b
- **2024/25 resit:** Q1a,b,c, Q2a,b,c, Q3b,c, Q4c, Q5c

> **IMPORTANT NOTE**
>
> The lecturer also flagged these as topics that must not be skipped even if older papers test them less: **Bastion host**, **Event Notification**, **Event Carried State Transfer**, **Network peering**, **3-tier architecture**, and **SQL injection**.

Starter Glossary

Read these before anything else. They appear in almost every section of this guide.

Term	Plain-English meaning
Distributed system	One system on the outside; many cooperating services on the
Monolith	A single application where all logic is packaged and deployed a
Microservice	A small, independently deployable service responsible for one
API	A defined set of rules that lets one program communicate with
Endpoint	A specific URL + method combination, e.g. `GET /books/42` .
API gateway	A single front door that routes client requests to the correct ba
Synchronous	The caller waits for a response before doing anything else.
Asynchronous	The caller sends a request and moves on — the response com
Middleware	Code that runs between receiving a request and sending a res
Functional requirement	A statement of what the system must do, such as allowing a us
Non-functional requirement	A statement of how well the system must perform, such as bei
3-tier architecture	A common split into presentation tier, application/business-log
Container	A lightweight, isolated runtime that packages an app with its d
IaaS / PaaS / SaaS	Three levels of cloud service — from "we manage the hardwar

Authentication (AuthN)	Proving who you are (login).
Authorisation (AuthZ)	Deciding what you are allowed to do once authenticated.
Encryption	Scrambling data so only the intended recipient can unscrambl
Hash	A fixed-length fingerprint of data — any change to the input pr
TLS	Transport Layer Security — the protocol that makes HTTPS se
JWT	JSON Web Token — a compact, signed token used to carry ide
Queue vs Event	A queue holds work to be done; an event records that somethi
Event Notification	An event pattern where the message mainly tells consumers th
Event Carried State Transfer (ECST)	An event pattern where the event carries the changed state its
Pod	The smallest deployable unit in Kubernetes — contains one or
CIDR	A notation like `/24` that describes a block of IP addresses.
Bastion host	A controlled jump box used by administrators to reach machin
Network peering	A private link between separate virtual networks so they can c
SQL injection	An attack where unsafe user input changes the meaning of an

1. Cryptography Foundations

CRITICAL — read before Section 2 (TLS)

Before TLS makes sense, you need to understand the building blocks it uses. This section covers all of them from scratch. If you understand this, the TLS handshake in Section 2 will feel logical rather than mysterious.

1.1 Hashing

A **hash function** takes any input (a word, a file, a contract) and produces a fixed-length string of characters called a **digest** or **hash**.

> **THREE RULES OF HASHING**
>
> 1. **Deterministic:** Same input always produces the same output.
> 2. **One-way:** You cannot reverse a hash to get the original input.

3. **Avalanche effect:** Change one character in the input → completely different hash output.

Example using SHA-256 (a common hash algorithm):

```
SHA256("hello") → 2cf24dba5fb0...44b
SHA256("hellp") → completely different string
```

Uses: Checking file integrity. Password storage (store hash, not plain text). Part of digital signatures and TLS.

1.2 Symmetric Encryption

In symmetric encryption, the **same key** is used to both encrypt and decrypt. Think of a padlock where you and your friend both have identical copies of the key.

Common algorithm: **AES (Advanced Encryption Standard)**.

WHY IS IT USED IN TLS?

Symmetric encryption is very fast, so after the TLS handshake establishes a shared secret, *all* actual data in the session is encrypted symmetrically. The problem is agreeing on a shared key securely — that is solved by asymmetric encryption.

KEY PROBLEM

If Alice wants to send Bob a message using symmetric encryption, she needs to send him the key first. But if an attacker is watching the network, they can steal the key. This is the fundamental problem asymmetric encryption solves.

1.3 Asymmetric Encryption (Public/Private Key)

Asymmetric encryption uses a **pair of mathematically linked keys**: a **public key** (you can share with anyone) and a **private key** (you keep secret, always).

Operation	Which key?	Who does it?	Purpose
Encrypt a message	Recipient's PUBLIC key	Anyone	Only the recipient (who has the priva
Decrypt a message	Your own PRIVATE key	Only you	Read the encrypted message sent to
Sign a message	Your own PRIVATE key	Only you	Prove the message came from you
Verify a signature	Sender's PUBLIC key	Anyone	Confirm the signature is genuine

> **THIS IS CONFUSING — HERE IS THE SIMPLE RULE**
>
> **For secrecy:** encrypt with the recipient's PUBLIC key (only they can decrypt with their private key).
> **For identity proof:** sign with YOUR PRIVATE key (anyone can verify with your public key, but only you could have signed it).

1.4 Diffie-Hellman Key Exchange

Diffie-Hellman (DH) is a way for two parties to **agree on a shared secret over a public channel** without ever sending the secret itself.

> **PAINT MIXING ANALOGY**
>
> Imagine Alice and Bob both start with the same public colour (yellow). Alice mixes in her secret colour (red → orange). Bob mixes in his secret colour (blue → green). They swap their mixed colours publicly. Alice adds her secret to Bob's mixture; Bob adds his secret to Alice's mixture. Both end up with the same final colour — but anyone watching only saw yellow, orange, and green, never the secrets.

In TLS, ECDHE (Elliptic Curve Diffie-Hellman Ephemeral) is used to generate the session keys so that even if the server's private key is stolen later, past sessions cannot be decrypted. This property is called **forward secrecy**.

1.5 HMAC — Hash-based Message Authentication Code

An HMAC combines a hash with a **secret key**. It produces a code that proves both:

- **Integrity:** the message has not been changed.
- **Authenticity:** the sender knew the secret key.

A plain hash alone only proves integrity — anyone can recompute the hash after modifying a message. An HMAC cannot be forged without the secret key.

In TLS: HMAC keys (called MAC keys) are used to authenticate each record sent during a session.

1.6 Digital Signatures — The Most Exam-Tested Cryptography Topic

A digital signature gives three guarantees that examiners test repeatedly:

- **Authenticity:** the message definitely came from the claimed sender.
- **Integrity:** the message was not changed in transit.
- **Non-repudiation:** the sender cannot later deny having sent it.

HOW TO CREATE AND VERIFY A DIGITAL SIGNATURE — STEP BY STEP

Signing (sender — e.g. Bob):

1. **Hash the message** using SHA-256 (or similar) → produces a digest.
2. **Encrypt the digest with Bob's private key** → this encrypted digest is the *digital signature*.
3. **Attach the signature to the message** and send both.

Verifying (recipient — e.g. Alice):

1. **Decrypt the signature using Bob's public key** → recovers the original digest.
2. **Hash the received message** independently.
3. **Compare the two hashes.** If they match → the signature is valid. The message came from Bob (only he has his private key) and was not modified.

WHY NON-REPUDIATION WORKS

Only Bob possesses his private key. If Alice can verify a signature using Bob's public key, only Bob could have created it. Bob cannot later claim "I didn't sign that" because no one else could have produced that signature.

QUICK QUIZ — CRYPTOGRAPHY

1. What three properties make a hash function useful for security?
2. What is the key problem with symmetric encryption when two strangers want to communicate?
3. Alice wants to send Bob a secret message. Which key does she use to encrypt it?
4. What is the difference between hashing and HMAC?
5. Name the three guarantees a digital signature provides.

▶ Show Answers

2. TLS, HTTPS & Digital Certificates

CRITICAL — appears in all 6 papers

This is the single most tested topic in the module. Every exam paper has at least one question on TLS, certificates, or digital signatures. Learn this section very well.

2.1 What HTTPS Gives You

Property	Meaning	How TLS achieves it
Confidentiality	No one can read your data in transit	Symmetric encryption (AES) usin
Integrity	Data was not altered in transit	HMAC (MAC keys) on every recor
Authenticity	You are talking to the real server, not an impostor	Server proves identity via digital c

2.2 Digital Certificates

A digital certificate is an electronic document that binds a server's **public key** to its **identity** (domain name). It is issued and signed by a **Certificate Authority (CA)**.

Key contents of an X.509 certificate

- **Subject:** who the certificate belongs to (e.g. `CN=www.mysite.com`)
- **Issuer:** which CA signed it (e.g. Let's Encrypt, DigiCert)
- **Public key:** the server's public key — clients use this during the handshake
- **Validity period:** not-before and not-after dates
- **CA's digital signature:** the CA's hash of all the above fields, signed with the CA's private key
- **Serial number** and **fingerprint**

How a certificate is created (CSR process)

1. The server generates a public/private key pair and keeps the private key secret.
2. The server creates a **Certificate Signing Request (CSR)** containing its public key and identity information.
3. The CSR is sent to a Certificate Authority (CA).
4. The CA verifies that the applicant controls the domain (e.g. by checking a file on the web server or a DNS record).

5. The CA signs the certificate with its own private key and returns the signed certificate.
6. The server installs the signed certificate.

Chain of Trust

Browsers do not trust individual servers directly — they trust a small set of **Root CAs** (pre-installed in the OS/browser). A chain works like this:

CERTIFICATE CHAIN (READ BOTTOM TO TOP)

Root CA (trusted by browser) → signs → Intermediate CA → signs → Server Certificate

The browser walks up the chain: it verifies the server cert is signed by the intermediate, and the intermediate is signed by a trusted root. If any link in the chain fails, the connection is rejected.

WHY USE INTERMEDIATE CAS?

Root CA private keys are kept offline in vaults. Intermediate CAs are used for day-to-day signing so that if an intermediate is compromised, it can be revoked without compromising the root.

How the browser validates a certificate (exam favourite)

1. Receive the server certificate and any intermediate certificates.
2. Check the certificate's **validity period** — has it expired?
3. Check the **domain name** in the certificate matches the URL being visited.
4. Verify each certificate's **CA signature** by decrypting the signature using the CA's public key and comparing against the certificate's hash.
5. Walk up to a **trusted root CA** — if the chain leads to a root CA that the browser trusts, the certificate is accepted.
6. Optionally check **revocation** via CRL or OCSP.

2.3 The TLS Handshake — Full Step-by-Step

The TLS handshake is what happens before any HTTPS data is exchanged. It establishes a **shared secret** and generates the **session keys** used to encrypt the connection.

EXAM NOTE

Questions say "explain the TLS handshake process from initial TCP connection through to encrypted data in transit". You need to know all the steps below, what is being sent, by whom, and why. Mark your steps clearly — examiners award marks per step.

1. **TCP Handshake (SYN / SYN-ACK / ACK)** — Before TLS starts, a plain TCP connection is established. This is not TLS — it is just the transport layer connecting.
2. **ClientHello** — The client sends: the highest TLS version it supports, a list of cipher suites it can use (e.g. ECDHE-AES256-GCM-SHA384), and a **client random** (32 bytes of random data).
3. **ServerHello** — The server responds with: the TLS version chosen, the cipher suite selected from the client's list, and a **server random** (32 bytes).
4. **Server Certificate** — The server sends its digital certificate (containing the server's public key and identity, signed by a CA), plus any intermediate certificates.
5. **ServerHelloDone** — The server signals it has finished its handshake messages.
6. **Client validates the certificate** — The browser checks: is it expired? Does the domain match? Is the CA signature valid? Is the chain trusted? If any check fails, the user sees a browser warning and the connection stops.
7. **Key Exchange** — The client and server use the agreed cipher suite to establish a shared secret:
   — With **ECDHE**: client and server exchange DH public values; both compute the same pre-master secret independently.
   — With **RSA key exchange** (older): client generates a pre-master secret, encrypts it with the server's public key, sends it. Only the server can decrypt with its private key.
8. **Both sides derive the Master Secret** — Using: pre-master secret + client random + server random, both sides independently compute the same master secret. They never send it — they each calculate it.
9. **Both sides derive the 4 Session Keys** — From the master secret, both sides independently derive four keys (see below).
10. **Client sends ChangeCipherSpec** — Signals: "I am now switching to encrypted communication using the session keys."

11 **Client sends Finished** — First encrypted message; contains a hash of the entire handshake to verify nothing was tampered with.

12 **Server sends ChangeCipherSpec + Finished** — Server does the same. Once both Finished messages are verified, the handshake is complete. All subsequent data is encrypted.

2.4 The Four Session Keys

This is a direct exam question in the 2023/24 refer/defer paper: *"state how many there are and what each is used for."*

Key name	Used by	Purpose
client_write_encryption_key	Client	Encrypts data sent FROM the client TO the server
server_write_encryption_key	Server	Encrypts data sent FROM the server TO the client
client_write_MAC_key	Client	Creates HMAC to authenticate records sent FROM client TC
server_write_MAC_key	Server	Creates HMAC to authenticate records sent FROM server T

WHY FOUR KEYS AND NOT ONE?

Using separate keys for each direction (client→server vs server→client) and separate keys for encryption vs authentication (MAC) provides stronger security. If one key were somehow compromised, it would not expose all traffic in both directions.

2.5 Asymmetric Encryption Example: Alice Sends a Secret Message to Bob

This specific scenario appeared in the 2023/24 main exam (7 marks):

MODEL APPROACH — ALICE SECURELY SENDS TO BOB, WITH ASSURANCE IT CAME FROM ALICE

1. Alice obtains Bob's **public key** (from his certificate or a key server).
2. Alice **encrypts the message with Bob's public key** — only Bob can decrypt this with his private key, ensuring confidentiality.
3. Alice **hashes the message** (SHA-256), then **encrypts the hash with her own private key** — this is her digital signature.
4. Alice sends: encrypted message + digital signature.
5. Bob **decrypts the message with his private key** — he can now read the content.

6. Bob **decrypts Alice's signature with Alice's public key** — this gives him the original hash.
7. Bob **hashes the decrypted message** himself and compares the two hashes.
8. If they match: the message was not tampered with (integrity) and only Alice could have signed it (authenticity).

QUICK QUIZ — TLS & CERTIFICATES

1. What is a CSR and why is it needed?
2. What is the chain of trust in the context of digital certificates?
3. Name the four session keys in TLS and state what each does.
4. What is the purpose of the client random and server random in the TLS handshake?
5. Why does forward secrecy (ECDHE) matter?

▶ Show Answers

15-MARK MODEL ANSWER: "EXPLAIN THE TLS HANDSHAKE FROM TCP CONNECTION TO ENCRYPTED SESSION"

Use this structure in the exam. Each numbered step = approximately 1 mark.

First, a TCP three-way handshake (SYN/SYN-ACK/ACK) establishes the underlying transport connection. The TLS handshake then begins. The client sends a ClientHello containing its supported TLS version, a list of cipher suites, and a client random value. The server responds with a ServerHello selecting the TLS version and cipher suite, along with a server random value. The server then sends its digital certificate, which contains its public key and is signed by a Certificate Authority. The client validates the certificate by checking it has not expired, the domain matches, and the CA's signature is valid by walking up the chain of trust to a trusted root CA. Key exchange then occurs — using ECDHE, both sides exchange DH public values and independently compute the same pre-master secret. Both sides then derive a master secret using: pre-master secret + client random + server random (using a pseudo-random function). From the master secret, both sides independently derive four session keys: client and server write encryption keys (for AES symmetric encryption in each direction) and client and server write MAC keys (for HMAC authentication of each record). The client sends a ChangeCipherSpec message followed by an encrypted Finished message. The server sends its own ChangeCipherSpec and Finished. Once both Finished messages are verified, all subsequent HTTP data is encrypted and authenticated using the session keys.

3. IAM: OAuth, OIDC, JWT & SSO

CRITICAL — flow question risk

Identity questions often look harder than they really are because the words sound similar. The exam becomes much easier once you separate **who the user is**, **what they can access**, and **which token carries the result**.

3.1 Quick term map

Term	Plain-English meaning	Exam note
Authentication	Proving who the user is	Think login
Authorisation	Deciding what the user may do	Think access rights
SSO	One login used across several systems	User convenience + central control
OAuth2	Delegated authorisation	Mainly about access delegation
OIDC	Identity layer on top of OAuth2	OAuth + login identity
JWT	A compact token format	Usually header.payload.signature

> **MOST COMMON MARK-LOSING MISTAKE**
>
> Do **not** say "OAuth is the login standard". OAuth2 is mainly about delegated authorisation. **OpenID Connect** is the layer that adds identity information for authentication-style flows.

3.2 JWT in one minute

A JWT (JSON Web Token) usually has **three parts**: `header.payload.signature` . The payload carries claims such as user ID, audience or expiry. The signature lets the receiver check the token has not been tampered with.

> **KEEP THIS DISTINCTION CLEAN**
>
> **Signed** does not automatically mean **encrypted**. A normal JWT can be readable if you decode it. The important point is that the signature proves integrity.

3.3 OIDC Authorisation Code Flow — Full 12-Step Version

This is the flow you are most likely to need in a sequence-diagram style answer. The 2023/24 main exam asked for this with a diagram worth 6 marks and an explanation worth 9 marks. You need to name the four actors clearly: **User, Browser, Application Server, Identity Provider (IdP)**.

THE FOUR ACTORS — DRAW THESE AS COLUMNS IN YOUR SEQUENCE DIAGRAM

User → Browser → Application Server → Identity Provider (IdP)

1. **User requests a protected page** — e.g. navigates to `/dashboard` in the browser.
2. **App server detects the user is not logged in** and constructs a redirect URL to the IdP, including: `client_id`, `redirect_uri`, `scope=openid profile email`, `response_type=code`, and a `state` value (to prevent CSRF).
3. **Browser is redirected to the IdP login page** (e.g. Google, Azure AD).
4. **User logs in at the IdP** — this is where authentication actually happens. The IdP verifies credentials.
5. **User grants consent** if prompted — the IdP asks whether the user allows the app to access their profile/email.
6. **IdP redirects the browser back to the app's `redirect_uri`** with a short-lived **authorisation code** in the URL query string, plus the `state` value.
7. **App server validates the `state`** to confirm this is a legitimate response, not a CSRF attack.
8. **App server makes a server-to-server POST request to the IdP's token endpoint** — exchanging the authorisation code for tokens. This is a back-channel request (browser not involved), including the `client_secret`.
9. **IdP returns tokens** — an **ID token** (identity info), an **access token** (for calling APIs), and optionally a **refresh token**.
10. **App server validates the ID token** — checks the JWT signature using the IdP's public key, verifies `iss` (issuer), `aud` (audience), and `exp` (expiry).
11. **App server creates a session** for the user and stores the tokens server-side. A session cookie is set in the browser.

12 **User is redirected to the originally requested protected resource**. The app uses the name, email and photo from the ID token payload to personalise the page.

TOKEN ROLES

- **ID token:** a JWT containing identity claims — who the user is (name, email, photo). Used for authentication.
- **Access token:** used by the app to call protected APIs or resource servers on the user's behalf. Used for authorisation.
- **Refresh token:** used to obtain new access/ID tokens when they expire, without making the user log in again.

WHY USE A CODE EXCHANGE RATHER THAN SENDING TOKENS DIRECTLY TO THE BROWSER?

If tokens were returned in the browser redirect URL, they could be stolen from browser history, logs or referrer headers. The code exchange happens server-to-server (back-channel), keeping tokens away from the browser.

3.4 JWT Deep Dive

A JWT is three Base64URL-encoded sections separated by dots: `header.payload.signature`

Part	Contents	Example
Header	Algorithm and token type	`{"alg":"RS256","typ":"JWT"}`
Payload	Claims about the user and session	`{"sub":"123","email":"z@x.com",'`
Signature	Hash of header+payload signed with private key	Verified using IdP's public key

Important JWT claims to know:

- `sub` — subject (unique user ID at the IdP)
- `iss` — issuer (which IdP issued this token)
- `aud` — audience (which app this token is intended for)
- `exp` — expiry time (Unix timestamp)
- `iat` — issued-at time
- `nonce` — links the token to a specific auth request (anti-replay)
- `email`, `name`, `picture` — profile claims

SIGNED ≠ ENCRYPTED

> A standard JWT (JWS) is signed but NOT encrypted. Anyone who intercepts it can read the payload. The signature only proves it was not tampered with. For sensitive payloads use JWE (JSON Web Encryption).

3.5 Full IAM Terminology — 2022/23 Reassessment Style

The 2022/23 reassessment Q5 asked for definitions of 12 IAM terms (2 marks each). Know these cold:

Term	Definition
Authentication	Verifying the identity of a user or system (proving who yo
Multi-Factor Authentication (MFA)	Requiring two or more independent factors to authentica
Authorisation	Determining what an authenticated user is permitted to d
Security Principal	Any entity (user, app, service) that can be authenticated
Single Sign-On (SSO)	One login grants access to multiple related systems witho
Active Directory (AD)	Microsoft's directory service for managing users, comput
Active Directory Federated Services (ADFS)	Extends AD authentication across organisational boundar
SAML	XML-based standard for exchanging authentication and a
OAuth 2.0	Framework for delegated authorisation — lets a user grar
OpenID Connect (OIDC)	Identity layer built on top of OAuth2 that adds authentica
JWT	Compact, URL-safe token format structured as header.pa
Identity Provider (IdP)	The system that authenticates users and issues tokens —
Service Provider (SP)	The application that relies on the IdP to authenticate user

3.6 How it appears in past papers

- 2022/23 reassessment Q5 — full 25 marks on IAM terminology definitions.
- 2022/23 main Q4 — OIDC authentication flow diagram and explanation (15 marks).
- 2023/24 main Q5 — JWT structure and use-case (5 marks).
- 2023/24 main Q6 — full OIDC sequence diagram + explanation (15 marks).

> **QUICK QUIZ — IAM**

1. What is the difference between authentication and authorisation?
2. Why does the OIDC flow use a short-lived authorisation code rather than sending tokens directly in the browser redirect?
3. What are the three parts of a JWT and what does each contain?
4. What is the difference between OAuth2 and OIDC?
5. What does the `aud` claim in a JWT represent and why does it matter?

▶ **Show Answers**

MODEL ANSWER: 9-MARK OIDC EXPLANATION (2023/24 MAIN STYLE)

The user requests a protected resource. The application detects there is no active session and redirects the browser to the Identity Provider with the client ID, redirect URI, scope (openid profile email) and a state value. The user authenticates at the IdP (this is where authentication happens). The IdP redirects the browser back to the application's redirect URI with a short-lived authorisation code. The application validates the state value to prevent CSRF, then makes a server-to-server POST request to the IdP's token endpoint, exchanging the code plus its client secret for tokens. The IdP returns an ID token and access token. The application validates the ID token: it checks the JWT signature using the IdP's public key, and verifies the issuer, audience and expiry claims. A server-side session is created, a session cookie is set in the browser, and the user is granted access to the protected resource. The name, email and photo from the ID token payload are used to personalise the page.

4. Monolith vs Microservices

CRITICAL — appears constantly

This is one of the safest revision topics because it comes up again and again. The exam rarely wants "microservices are better". It usually wants a **balanced trade-off answer** with technical and business reasoning.

4.1 The two shapes in simple terms

Architecture	Simple meaning	Main strength
Monolith	One main application deployed as one unit	Simplicity
Microservices	Many smaller services cooperating over a network	Agility and independent scaling

4.2 What "loose coupling" means

Loose coupling means one service depends as little as possible on the internal details of another. If the payments service changes its internal code, the orders service should not break as long as the agreed interface stays stable.

> **EASY MEMORY LINE**
>
> Loose coupling = **"I know how to call you, not how you work inside."**

4.3 Why businesses move towards microservices

- Different teams can work and deploy independently.
- Busy services can scale without scaling the whole system.
- One failing service does not always bring down everything else.
- Different parts of the system can evolve at different speeds.

4.4 Functional vs non-functional requirements

Functional requirements describe **what the system must do**. **Non-functional requirements** describe **how well it must do it**.

Requirement type	Simple meaning	Examples
Functional	Features or behaviours the system must provide	User can log in, customer can
Non-functional	Quality conditions or constraints on the system	Fast response time, high avail

> **FAST MEMORY LINE**
>
> Functional = **what it does**. Non-functional = **how well it does it**.

4.5 Why monoliths still matter

> **DO NOT THROW MONOLITHS AWAY TOO QUICKLY**
>
> For a small, stable system a monolith can be the smarter choice because it is simpler to build, test, deploy and debug. A strong answer says **when the monolith is sensible**, not just when microservices are attractive.

4.6 3-tier architecture

A basic **3-tier architecture** separates the system into three main layers:

1. **Presentation tier** — the web UI or client.
2. **Application tier** — the business logic / API.
3. **Data tier** — the database.

```
Browser / UI
     |
App / API / server logic
     |
Database
```

This is not exactly the same thing as microservices, but it is still a very useful architecture pattern and it appears in the current slide deck. It is especially useful when explaining separation of concerns and network placement such as web/app/database subnets.

4.7 API Gateway in this topic

An **API gateway** is a single front door in front of several backend services. It can route requests, centralise authentication, apply rate limiting and sometimes combine results from multiple services into one response.

> **TRADE-OFF**
>
> The gateway itself becomes important infrastructure, so it must be scalable and resilient. Otherwise it becomes a bottleneck or single point of failure.

> **5-MARK API GATEWAY ANSWER SHAPE**
>
> State that it is a single entry point, then give 2-3 benefits such as routing, centralised security, rate limiting or response aggregation, and finish with one trade-off such as bottleneck or single point of failure risk.

4.8 How it appears in past papers

- 2022/23 reassessment compared monolith and microservice approaches.
- 2023/24 asked for properties of a microservice and a composite microservice.
- 2024/25 resit asked for business benefits of a distributed microservice architecture.
- 2024/25 resit also asked directly about functional vs non-functional requirements.
- API gateway reasoning appears repeatedly as a design justification question.

> **QUICK QUIZ — ARCHITECTURE**

1. What makes a distributed system still feel like one system to the user?
2. What is the biggest operational advantage of a monolith for a small stable system?
3. Why can microservices scale more cheaply in some cases?
4. What does loose coupling mean in simple terms?
5. What is one risk of splitting a system into many services?

▶ Show Answers

HIGH-MARK STRUCTURE FOR COMPARE QUESTIONS

Use the same headings for both sides: **deployment, scaling, team independence, resilience, data consistency, operational complexity**. End with a judgement about which is better for the scenario given.

5. APIs: REST, Endpoints & JSON

HIGH — design question potential

API questions are usually very manageable marks if you stay practical. The examiner normally wants a **clean endpoint**, the **right HTTP method**, and a short explanation of why the design makes sense.

5.1 Core building blocks

Term	Meaning	Example
API	The rules that let one program talk to another	A web app calling a backend servi
Endpoint	A specific method + URL combination	`GET /books/42`
JSON	Text format for structured data	`{"name":"Ava","age":20}`
Path parameter	Identifies the main resource	`/books/42`
Query parameter	Filters or narrows results	`?author=Orwell`

5.2 Method choice

- **GET** = retrieve data
- **POST** = create something new
- **PUT** = replace a resource
- **PATCH** = partially update a resource

- **DELETE** = remove a resource

> **SIMPLE DESIGN RULE**
>
> **Path points to the thing.**
> **Query qualifies the thing.**

5.3 Example endpoint design

A sensible bookstore endpoint could be:

```
GET /books/hardback/dystopian?author=George%20Orwell&title=1984&maxPrice=10.99
```

Here, the path gives the main resource context and the query adds optional filters.

5.4 JSON quick-practice

JSON questions are usually easy marks if you stay calm and focus on **keys**, **values**, **objects** and **arrays**.

```
{
  "age": 25,
  "firstName": "Bob",
  "lastName": "Martin",
  "children": ["Jim", "Jane", "Susan"]
}
```

> **QUICK JSON RULES**
>
> Use double quotes for keys and string values. Use square brackets for a list. Do not put random prose into the structure.

5.5 REST and HATEOAS

A REST-style API normally uses predictable resource-oriented URLs and standard HTTP methods. **HATEOAS** is a stricter idea where responses include links to what the client can do next.

> **EXAM-SAFE WORDING**
>
> If you mention HATEOAS, keep it simple: **the response helps guide the client to valid next actions**.

5.6 SQL injection — new-year priority

SQL injection happens when unsafe user input is mixed directly into SQL so the input changes the meaning of the query.

```
Bad idea:
SELECT * FROM users WHERE username = ' " + userInput + " '
```

WHY IT MATTERS

An attacker may read data they should not see, change data, or bypass login checks if the query is built unsafely.

HOW TO PREVENT IT

- Use parameterised queries / prepared statements.
- Validate input sensibly.
- Use least-privilege database accounts.
- Do not concatenate raw user input into SQL strings.

5.7 How it appears in past papers

- 2024/25 main asked for endpoint design and example calls.
- 2023/24 refer/defer asked for an API URL returning banking transactions.
- 2022/23 reassessment included HATEOAS as an examinable extra.
- The lecturer's revision strategy explicitly lists SQL injection as a new-year topic to know thoroughly.

QUICK QUIZ — API BASICS

1. What is the difference between a path parameter and a query parameter?
2. Why is JSON so common in web APIs?
3. Which HTTP method is most suitable for retrieving data only?
4. When might you use PATCH instead of PUT?
5. What is the safest main way to prevent SQL injection?

▶ Show Answers

EXAM ANSWER PATTERN

> For an endpoint design question: **state method -> write endpoint -> explain path vs query split -> give one example call -> mention JSON response**. For a SQL injection question: **define the attack -> explain the risk -> state prevention using parameterised queries**.

6. Express, Middleware & CORS

HIGH — practical explanation marks

These questions are usually about **flow**. If you can explain what happens to a request in the right order, you are in a strong position.

6.1 Express request flow

1. **A request arrives** with a method, path, headers and maybe a body.
2. **Middleware runs in registration order**.
3. **Each middleware either ends the response or calls `next()`**.
4. **The matching route handler runs**.
5. **The server sends a response** with a status code and optional body.

6.2 What middleware is for

- Logging requests
- Checking authentication
- Validating input
- Parsing JSON
- Applying CORS policy

```
app.use((req, res, next) => {
  console.log(req.method, req.path);
  next();
});
```

> **MEMORY LINE**
>
> Middleware = **code that runs before the final endpoint response is sent**.

6.3 Response codes you should know

Code	Meaning	Typical use
200	OK	Successful GET or update
201	Created	Successful POST that created a resource
400	Bad Request	Invalid input from client
401	Unauthorised	Missing or invalid authentication
404	Not Found	Route or resource does not exist
500	Internal Server Error	Unexpected server-side failure

6.4 Same-origin policy and CORS — full exam coverage

The 2022/23 main, 2023/24 refer/defer, and 2024/25 resit papers all tested CORS. The resit paper awarded **10 marks** for it. You need more than a one-liner here.

What an origin is

An **origin** is the combination of **scheme + domain + port**. All three must match for two URLs to be considered the same origin.

URL A	URL B	Same origin?	Why
https://myapp.com	https://myapp.com/api/books	Yes	Same scheme, domain, port (4
https://myapp.com	http://myapp.com	No	Different scheme
https://myapp.com	https://api.myapp.com	No	Different subdomain
https://myapp.com	https://myapp.com:8080	No	Different port

Same-origin policy

Browsers enforce a rule: JavaScript on `https://myapp.com` cannot read responses from `https://api.other.com` unless that server explicitly permits it. This protects users — without it, a malicious page could silently read your bank data using your logged-in session.

How CORS works — preflight request

1. A browser script on `https://frontend.com` tries to call `https://api.backend.com/data` .
2. For non-simple requests (e.g. using `PUT` , custom headers, or JSON content-type), the browser first sends an **HTTP OPTIONS preflight request** to the server asking: "do you

allow cross-origin requests from this origin with this method?"

3 The server responds with **CORS headers** stating what it allows.

4 If the server allows it, the browser proceeds with the real request. If not, the browser blocks it and the JavaScript sees an error.

Key CORS response headers

Header	What it controls	Example valu
`Access-Control-Allow-Origin`	Which origins are allowed	`https://mya`
`Access-Control-Allow-Methods`	Which HTTP methods are allowed	`GET, POST,`
`Access-Control-Allow-Headers`	Which request headers are allowed	`Content-Typ`
`Access-Control-Allow-Credentials`	Whether cookies/auth headers are allowed	`true`

Handling CORS in Node.js Express — three levels

The 2022/23 main exam explicitly asked: how does your Node.js server code allow CORS, and how do you limit which resources are shareable?

```
// Level 1: Allow ALL origins (open — use only in dev/internal APIs)
const cors = require('cors');
app.use(cors());

// Level 2: Allow a specific origin only
app.use(cors({
  origin: 'https://myapp.com',
  methods: ['GET', 'POST'],
  allowedHeaders: ['Content-Type', 'Authorization']
}));

// Level 3: Allow CORS only on a specific route and method
app.get('/public/data', cors({ origin: 'https://myapp.com' }), (req, res) => {
  res.json({ data: 'ok' });
});
```

EXAM-SAFE HIERARCHY TO MENTION

1. "Allow all" — `cors()` with no options. Simplest, least secure.
2. "Allow specific origin and methods" — pass an options object to `cors()` .

3. "Restrict per route" — apply `cors()` as per-route middleware rather than globally.

SERVER-TO-SERVER AS AN ALTERNATIVE

CORS is a browser restriction. If your frontend calls your own backend, and your backend calls the third-party API, the browser never makes the cross-origin call directly. This avoids CORS entirely — the server-to-server call is unrestricted by CORS rules.

6.5 How it appears in past papers

- 2023/24 asked for the concept and operation of Express middleware.
- 2024/25 main asked how Express handles requests so the right endpoint runs.
- 2022/23 main and refer/defer papers tested CORS directly.

QUICK QUIZ — EXPRESS & CORS

1. What does `next()` do in Express middleware?
2. Why does middleware order matter?
3. Give one situation where a server should return 404.
4. What is an origin in web security terms?
5. Why does the browser block some cross-origin requests by default?

▶ Show Answers

EXAM ANSWER SKELETON

If the question mentions Express, explain the **ordered request flow**. If it mentions CORS, define **origin** and explain that the **browser checks the server's policy** before allowing the request.

7. API Documentation & OpenAPI

MEDIUM — easy marks if prepared

Documentation questions are less flashy than security or messaging, but they are reliable marks because the logic is straightforward: **a good API contract is only useful if people can understand and use it correctly**.

7.1 Why documentation matters

- It stops teams guessing paths, methods and request bodies.
- It reduces integration mistakes and rework.
- It helps frontend, backend and test teams work in parallel.
- It makes onboarding and long-term maintenance easier.

7.2 OpenAPI core structure

Element	What it describes
Info	Basic API details such as title and version
Servers	Where the API lives
Paths	The endpoints and methods
Components	Reusable schemas, parameters and security definitions
Tags	Logical grouping of endpoints

7.3 Three documentation approaches

> **DESIGN-FIRST**
>
> Start by modelling the API before implementation. Good when many people need to agree the shape early.

> **API-FIRST**
>
> Start with a formal API contract, often OpenAPI. Good for parallel frontend/backend/test work.

> **CODE-FIRST**
>
> Generate or build docs from working code. Often practical for smaller teams where the implementation is moving quickly.

7.4 HATEOAS and Richardson's Maturity Model

The 2022/23 reassessment asked a **13-mark question** on HATEOAS that required: an explanation, an opinion on whether all REST APIs conform, a JSON example, an advantage, a disadvantage, and a reference to Richardson's Maturity Model. Do not treat this as a minor topic.

What HATEOAS means

HATEOAS (Hypermedia As The Engine Of Application State) is a REST constraint where each API response includes links telling the client what actions are available next. Instead of the client needing to know all possible URLs upfront, the server guides it.

```
{
  "orderId": 123,
  "status": "pending",
  "_links": {
    "self": { "href": "/orders/123" },
    "cancel": { "href": "/orders/123/cancel", "method": "DELETE" },
    "pay":    { "href": "/orders/123/payment", "method": "POST" }
  }
}
```

The client now knows it can cancel or pay this order without hard-coding those URLs into the client code.

> **DO ALL REST APIS CONFORM TO HATEOAS?**
>
> **No — and this is the expected honest answer.** The vast majority of real-world REST APIs do not implement HATEOAS. Most are only at Richardson Level 2. True HATEOAS (Level 3) is the ideal described in Roy Fielding's original REST dissertation, but it is rarely implemented in practice because of the extra effort and the fact that most clients do not use the links anyway.

Richardson's Maturity Model — 4 levels

Level	Name	What it means
0	The Swamp of POX	One URL, one method (usually POST for everything)
1	Resources	Multiple URLs representing resources, but still using one r
2	HTTP Verbs	Correct HTTP methods (GET/POST/PUT/DELETE) used wit
3	Hypermedia Controls (HATEOAS)	Responses include links to next valid actions

> **MEMORY SHORTCUT**
>
> Level 0 = chaos. Level 1 = named resources. Level 2 = right verbs. Level 3 = HATEOAS. **Most real APIs are Level 2.**

HATEOAS advantage and disadvantage

ADVANTAGE

Clients are less brittle — they do not need to hard-code URLs. The server can change URLs and the client follows links dynamically. Promotes true loose coupling.

DISADVANTAGE

Significantly more complex to implement and document. Most clients ignore the links anyway, so the extra effort rarely pays off in practice. Larger response payloads.

7.5 How it appears in past papers

- 2022/23 reassessment asked about three documentation approaches using OpenAPI.
- 2023/24 refer/defer compared design-first, API-first and code-first.
- 2024/25 resit asked why documenting API endpoints matters and what OpenAPI usually contains.

QUICK QUIZ — API DOCUMENTATION

1. Why can poor API documentation slow a team down even if the API works?
2. What problem does OpenAPI solve?
3. When is design-first especially useful?
4. What is one advantage of code-first for a small team?
5. What is the basic idea of HATEOAS?

▶ Show Answers

WHAT A STRONG ANSWER SOUNDS LIKE

Do not just define the three approaches. Say **when** you would use each one and **why** it fits that team or project.

8. Messaging: Queues, Pub-Sub & Events

CRITICAL — appears in every paper set

Messaging is one of the most repeated exam themes. The safest way to remember it is to separate **ask-and-wait**, **hand-off work**, and **announce something happened**.

8.1 Quick mental map

Pattern	Simple meaning	Typical strength
HTTP	Ask now and wait for a reply	Simple and immediate
Message queue	Hand off work and continue	Resilience and bufferin
Pub-sub	One message or event sent to many interested consumers	Extensibility
Event-driven	Services react to facts that something happened	Loose coupling

> **ONE-LINE MEMORY AID**
>
> HTTP = ask and wait.
> Queue = hand off and continue.
> Pub-sub = tell many listeners.
> Event = a fact that happened, not a command.

8.2 Queue vs pub-sub

In a **shared queue**, one message is typically processed by one consumer. In **publisher-subscriber**, multiple subscribers can each receive the same event or message and react independently.

8.3 Thick and thin events

- **Thin event:** carries minimal data, often just an ID and timestamp. Consumers usually make another API call for the full state.
- **Thick event:** carries useful state data with it, so consumers may not need a follow-up call.

> **TRADE-OFF**
>
> Thin events reduce payload size but often increase follow-up API traffic. Thick events reduce follow-up calls but can duplicate more data and need careful schema design.

8.4 Event Notification vs Event Carried State Transfer (ECST)

Pattern	What the event carries	What consum
Event Notification	Minimal information such as ID and timestamp	Call another se

Event Carried State Transfer (ECST)	The changed state itself	Update local d

SIMPLE WAY TO REMEMBER IT

Event Notification says **"something happened"**.
ECST says **"something happened and here is the new state"**.

ECST is closely tied to **thick events**. Event Notification is closely tied to **thin events**.

8.5 RabbitMQ vs Kafka at a simple level

Technology	Best thought of as	Good for
RabbitMQ	Message broker with queues and exchanges	Reliable message delivery patterns
Kafka	Persistent event streaming platform	Replayable event streams and high-volu

8.6 How it appears in past papers

- 2022/23 main compared shared queue and pub-sub and also asked about event-driven data sharing.
- 2023/24 asked for a redraw from message-based to event-driven architecture.
- 2024/25 main asked for advantage, disadvantage and use-case for HTTP, queue and event messaging.
- Refer/defer asked about thick vs thin events and synchronous vs asynchronous approaches.
- The lecturer's revision strategy explicitly highlights Event Notification and ECST as new-year priorities.

QUICK QUIZ — MESSAGING

1. Why is HTTP usually described as synchronous between services?
2. What advantage does a message queue bring when the consumer is slow or temporarily unavailable?
3. How is pub-sub different from a single shared queue?
4. What is the difference between Event Notification and ECST?
5. Why can event-driven designs lead to eventual consistency?

▶ Show Answers

HIGH-MARK ANSWER SHAPE

> For pattern-choice questions, always tie the answer to **immediacy, coupling, resilience, number of consumers, and consistency**. For Event Notification vs ECST, explicitly say whether consumers need a follow-up API call.

9. Cloud: IaaS, PaaS, SaaS & Shared Responsibility

HIGH — common explain/compare topic

Cloud questions are usually really about **who manages what**, **how much control you keep**, and **why a business would choose one model over another**.

9.1 Service models

Model	What you get	Who manages more?
IaaS	Infrastructure such as VMs, storage and networking	You manage more
PaaS	A managed platform to deploy your application	Provider manages more of the platforn
SaaS	The finished software as a service	Provider manages almost everything
DBaaS	A managed database service	Provider manages the database platfo

9.2 Shared responsibility in plain English

Shared responsibility means the cloud provider does not do everything. The split changes depending on the service model. In IaaS you still manage much more. In PaaS you hand over more platform responsibility. In SaaS you mostly just use the software.

> **BUSINESS VALUE**
>
> - Lower up-front cost than buying lots of hardware.
> - Faster delivery of new environments.
> - Easier scaling and global reach.
> - Less day-to-day infrastructure work in more managed models.

9.3 When to choose which

- **IaaS:** choose it when you need detailed OS, network or runtime control.

- **PaaS:** choose it when you mainly want to deploy the app and let the provider handle more of the platform.
- **DBaaS:** choose it when you need database capability without running and patching DB servers yourself.

9.4 How it appears in past papers

- 2022/23 main asked about shared responsibility across cloud models.
- 2022/23 reassessment tested IaaS, PaaS, SaaS and DBaaS with use-cases.
- 2024/25 resit asked for IaaS vs PaaS and cloud business benefits.

QUICK QUIZ — CLOUD MODELS

1. What is the main difference between IaaS and PaaS?
2. Why can the cloud reduce capital expenditure?
3. What does shared responsibility mean?
4. Give one good use-case for DBaaS.
5. Why might a business choose PaaS over IaaS?

▶ Show Answers

EXAM-SAFE CONCLUSION

The more managed the service, the less you manage yourself. High-mark answers always connect that trade-off to a realistic use-case.

10. Resilience, Availability Zones & Elastic Compute

HIGH — diagram-friendly topic

This topic is about keeping a system running when demand changes or components fail. The exam often rewards very clear diagrams here.

10.1 Key distinctions

Term	Meaning
High availability	Staying up through expected failures

Resilience	Withstanding and recovering from wider failure
Availability zone (AZ)	A separate cloud location within a region used for fault tolerance
Load balancer	A front component that spreads traffic across healthy instances
Elastic compute	Scaling resources up or down with demand

10.2 Typical resilient architecture

```
Clients
   |
Load Balancer
  / \
AZ1 AZ2
 |   |
App App
```

The load balancer gives one front door. Multiple instances sit behind it, ideally across more than one availability zone. If one instance or one zone fails, traffic can continue going elsewhere.

10.3 Horizontal vs vertical scaling

- **Horizontal scaling** = add more instances.
- **Vertical scaling** = make one machine bigger.

> **EXAM PREFERENCE**
>
> Distributed-systems questions usually favour **horizontal** scaling because it fits resilience and elasticity better.

10.4 Availability Zone benefits — 6-mark model answer

The 2023/24 refer/defer Q2a asked: *"provide three clear benefits an availability zone offers regarding system resilience. Two marks are available for each justified benefit."* The marker rubric even gives you the answer format: **"An AZ offers [benefit] which offers protection of [thing] by [mechanism]."** Use that format exactly.

> **THREE AZ BENEFITS — WRITE IN THIS EXACT STRUCTURE (2 MARKS EACH)**
>
> **Benefit 1 — Fault isolation:** An availability zone offers physical separation from other zones in the same region, which offers protection against a single site failure (power cut, cooling failure, hardware outage) by ensuring that a failure in one AZ does not cascade to instances running in another AZ.

Benefit 2 — Continued service during partial outages: An availability zone offers independent infrastructure (independent power, networking and physical buildings), which offers protection against unplanned downtime by allowing a load balancer to route traffic away from a failed AZ to healthy instances in surviving zones without any manual intervention.

Benefit 3 — Geographic distribution within a region: An availability zone offers low-latency proximity to other AZs within the same region, which offers protection against data loss by enabling synchronous data replication between AZs so that if one AZ loses its database, another AZ's replica can take over with minimal or no data loss.

10.5 How it appears in past papers

- 2024/25 resit asked directly about the purpose and operation of a load balancer in elastic-compute solutions.
- 2023/24 refer/defer Q2a — 6-mark AZ benefits question with the specific "benefit X, protection of Y, by Z" rubric format.
- Older cloud design questions also rewarded highly available, elastically scalable diagrams.

QUICK QUIZ — RESILIENCE

1. What is the difference between high availability and resilience?
2. Why can a load balancer improve both performance and resilience?
3. What does an availability zone help protect you against?
4. What is horizontal scaling?
5. Why is elastic compute valuable to a business as well as technically useful?

▶ Show Answers

WHAT EXAMINERS WANT IN DIAGRAM ANSWERS

Show **one front door**, **multiple backend instances**, and ideally **more than one availability zone**. Then explain both normal traffic flow and what happens during failure.

11. Containers, Docker & Compose HIGH — practical deployment topic

Container questions usually reward a simple explanation of **what problem containers solve**, **how they differ from VMs**, and **how persistence/configuration are handled**.

11.1 VM vs container

Feature	Virtual machine	Container
OS model	Includes a full guest OS	Shares host kernel
Weight	Heavier	Lighter
Startup	Slower	Faster
Isolation	Stronger full-machine isolation	Process-level isolation

11.2 Image vs container

A **Docker image** is the packaged blueprint. A **container** is a running instance created from that image.

> **MEMORY LINE**
>
> Image = blueprint. Container = running instance.

11.3 Docker Compose, volumes and env vars

- **Compose** declaratively defines a multi-container setup in a single YAML file.
- **Named volumes** persist data outside the container's short-lived writable layer.
- **Environment variables** keep configuration such as ports, hostnames and secrets out of hard-coded source files.

> **IMPORTANT BEGINNER POINT**
>
> If a database writes only inside the container and the container is removed, the data is lost. That is the problem the **named volume** solves.

11.4 Annotated Docker Compose YAML — 2023/24 refer/defer Q5b style

The 2023/24 refer/defer exam gave a `compose.yaml` file and asked you to explain specific **line numbers** for 2 marks each (12 marks total). The question said: explain the *declaration* at that line — not the specific values, but what that block *achieves*. Here is a representative example with the same key declarations the exam tested:

```
# Line 2
services:
  api:
    image: myapp-api:latest
```

```
# Line 6
    ports:
      - "3000:3000"
# Line 8
    environment:
      - MYSQL_HOST=db
      - MYSQL_USER=root
      - MYSQL_PASSWORD=secret
# Line 14
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:3000/health"]
      interval: 30s
      retries: 3
# Line 20
    depends_on:
      db:
        condition: service_healthy
# Line 24
    volumes:
      - db_data:/var/lib/mysql

  db:
    image: mysql:8

volumes:
  db_data:
```

HOW TO ANSWER EACH LINE-REFERENCE QUESTION (2 MARKS EACH)

Line 2 — `services:` : The `services` declaration defines the individual containers that make up the application. Each named entry under it (e.g. `api` , `db`) represents one container that Compose will create and manage. This is the top-level block that tells Compose what to run.

Line 6 — `ports:` : The `ports` declaration maps a port on the host machine to a port inside the container. The format is `host:container` . This makes the container's service reachable from outside Docker — for example, a browser on the host can call the API on port 3000.

Line 8 — `environment:` : The `environment` declaration injects configuration values into the container as environment variables at runtime. This keeps sensitive values such as database credentials and hostnames out of the source code and the container image, and allows the same image to be used across different environments (dev/staging/prod) with different settings.

Line 14 — `healthcheck:` : The `healthcheck` declaration defines a command Compose runs periodically to determine whether the container is healthy and ready to serve traffic. If the check fails repeatedly (based on `retries`), the container is marked unhealthy. Other services

can use this status to decide whether to start (via `depends_on: condition: service_healthy`).

Line 20 — `depends_on:` : The `depends_on` declaration controls the startup order of services. With `condition: service_healthy` , the API container will not start until the `db` container passes its healthcheck. Without this, the API might try to connect to a database that is not yet ready and crash on startup.

Line 24 — `volumes:` : The `volumes` declaration mounts a named Docker volume into the container at the specified path. This persists data outside the container's ephemeral writable layer. For a database, this means the data files survive if the container is stopped, recreated or updated — without it, all stored data would be lost when the container is removed.

THE PATTERN FOR ANY LINE-REFERENCE ANSWER

1) Name what the declaration is. **2)** Explain what it achieves in the wider system. **3)** State what would happen without it. Two marks means: definition + consequence.

11.5 How it appears in past papers

- 2022/23 main compared VMs and containers.
- 2023/24 asked about environment variables in Compose and Kubernetes-style setups.
- 2024/25 resit asked directly about Docker named volumes.

QUICK QUIZ — CONTAINERS

1. What is the difference between a Docker image and a Docker container?
2. Why can containers start faster than virtual machines?
3. What problem does a named volume solve for a database?
4. What is one benefit of environment variables?
5. What is the difference between a type 1 and type 2 hypervisor?

▶ Show Answers

EXAM TIP

If the question asks why a named volume is useful, answer three things clearly: **what problem exists without it**, **how the volume solves that problem**, and **what the consequence is if you do not use one**.

12. Kubernetes & Rolling Updates

HIGH — current-syllabus favourite

Kubernetes questions are easier once you remember the object hierarchy. Most lost marks come from mixing up **pod**, **ReplicaSet**, **Deployment** and **Service**.

> **2023/24 MAIN EXAM — Q4 WAS WORTH 25 MARKS AND ASKED FOR ALL 8 COMPONENTS INDIVIDUALLY**
>
> That exam paper gave roughly 3 marks per component. You need a clear one-paragraph explanation of each one — not just a one-liner.

12.1 Cluster structure — understand this first

```
Kubernetes Cluster
├── Control Plane (the brain — manages the cluster)
│   ├── API Server
│   ├── Controller Manager
│   ├── Scheduler
│   └── etcd (key-value store)
└── Worker Nodes (where your app actually runs)
    ├── Kubelet (agent on each node)
    ├── Pods → Containers
    └── Services (networking)
```

12.2 All 8 components — exam-ready definitions

Component	What it does
Control Plane	The management layer of the cluster. Contains the API server, scheduler, controller ma
Kubelet	An agent that runs on every worker node. It watches for pod assignments from the con
Node	A physical or virtual machine that runs pods. A cluster has multiple nodes for availabilit
Pod	The smallest deployable unit in Kubernetes. A pod wraps one or more containers that s
ReplicaSet	Ensures a specified number of identical pod replicas are running at all times. If a pod c
Deployment	Manages ReplicaSets over time. Handles rolling updates (creates a new ReplicaSet for
ConfigMap	Stores non-sensitive configuration data as key-value pairs (e.g. environment variables,

Service	Provides a stable network endpoint to access pods. Because pods are ephemeral and

12.3 Service types — when to use which

Type	Access scope	When to use
ClusterIP	Internal to the cluster only	Microservice-to-microservice comm
NodePort	Exposed on a port on each node's IP	Testing or development — not recor
LoadBalancer	Externally accessible via a cloud load balancer	Production services that need to be

12.4 Rolling update — step by step

The 2024/25 main exam asked: *"explain how Kubernetes enables rolling updates with no downtime using a Deployment and ReplicaSet"* (13 marks). Use this structure:

1. **You update the Deployment** with a new container image version (e.g. `kubectl set image deployment/api api=myapp:v2`).
2. **The Deployment creates a new ReplicaSet** configured with the new image version.
3. **The new ReplicaSet starts creating new pods** one at a time (controlled by `maxSurge` setting).
4. **Each new pod must pass its readiness check** before it receives traffic. This ensures no broken pods go live.
5. **Once a new pod is healthy, one old pod is terminated** (controlled by `maxUnavailable` setting). At no point are all pods down simultaneously.
6. **This continues until the new ReplicaSet is at full count** and the old ReplicaSet is scaled to zero.
7. **The old ReplicaSet is retained** (not deleted) so a rollback can instantly scale it back up if needed.

> **WHY NO DOWNTIME?**
>
> Because Kubernetes never removes an old pod until at least one healthy new pod is ready to take its place. Traffic is gradually shifted, not cut over all at once.

12.5 How it appears in past papers

- 2023/24 main Q4 — **25 marks** — all 8 components defined individually including Service types (ClusterIP/NodePort/LoadBalancer).
- 2024/25 main Q3 — rolling updates with Deployment and ReplicaSet (13 marks).
- 2023/24 refer/defer Q5 — rolling upgrades explanation (7 marks).

QUICK QUIZ — KUBERNETES

1. What is the difference between a Pod and a ReplicaSet?
2. Why is a Deployment needed on top of a ReplicaSet?
3. What does a Kubelet do?
4. What problem does a ConfigMap solve?
5. When would you choose LoadBalancer over ClusterIP for a Service?

▶ Show Answers

13-MARK ROLLING UPDATE MODEL ANSWER STRUCTURE

1) Define what a ReplicaSet does (maintains desired pod count). **2)** Define what a Deployment adds (manages ReplicaSets, handles version changes). **3)** Walk through the rolling update: new ReplicaSet created → new pods start → readiness checks pass → old pods removed one at a time → old ReplicaSet retained for rollback. **4)** Explain why there is no downtime: healthy new pods receive traffic before old pods are terminated.

13. Data Layer, ACID & SAGA

HIGH — common scaling and transaction topic

Database questions often look varied, but they usually fall into three buckets: **keeping data correct**, **scaling reads and writes**, and **handling distributed transactions**.

13.1 Keeping data correct

Referential integrity protects relationships between related records, for example making sure an order still points to a valid customer. Common actions include **cascade**, **restrict** and **set null**.

13.2 Connection pooling and read replicas

- **Connection pool:** a reusable set of ready-made database connections.
- **Read replica:** a read-only copy of the primary database used to spread read traffic.

> **IMPORTANT LIMITATION**
>
> Read replicas help with **reads**. They do **not** remove the main write bottleneck, and they may briefly return stale data because of replication lag.

13.3 Sharding and consistent hashing

Sharding splits data across nodes. **Consistent hashing** is a strategy that reduces how much data must move when nodes are added or removed.

13.4 ACID

Letter	Property	Plain meaning
A	Atomicity	All steps in a transaction succeed together, or none of them are applied. No ha
C	Consistency	The database moves from one valid state to another. All integrity rules are pres
I	Isolation	Concurrent transactions cannot see each other's intermediate state. They beh
D	Durability	Once a transaction is committed, it survives crashes and power failures.

ACID works naturally inside one transactional database. The problem arises in microservices — each service has its own database, so a single cross-service ACID transaction is not directly possible.

13.5 The Distributed Transaction Problem

Imagine an e-commerce order that spans three services: **Order Service**, **Payment Service**, **Shipping Service**. In a monolith you could wrap all three operations in one database transaction. In microservices you cannot — each service controls its own database. If Payment succeeds but Shipping fails, you now have inconsistent data across services.

13.6 SAGA Pattern — Full Worked Example

The 2023/24 main exam asked for: *"explain the SAGA orchestration pattern, clearly showing a successful and a failed transaction spanning three microservices"* (10 marks). This is exactly the example you need.

> **CORE IDEA OF SAGA**
>
> Break one large distributed transaction into a sequence of **local transactions**, each within one service. If a later step fails, run **compensating transactions** to undo the earlier steps. You accept **eventual consistency** rather than strict ACID atomicity.

Successful transaction (happy path) — Order → Payment → Shipping

1. **Order Service** creates a new order with status "PENDING" and publishes an `OrderCreated` event.
2. **Payment Service** receives the event, charges the customer, marks payment as "COMPLETED", publishes `PaymentProcessed`.
3. **Shipping Service** receives the event, creates a shipment, marks it "DISPATCHED", publishes `ShipmentCreated`.
4. **Order Service** receives `ShipmentCreated`, updates order status to "CONFIRMED". Transaction complete.

Failed transaction (compensating path) — Shipping fails

1. **Order Service** creates order "PENDING", publishes `OrderCreated`.
2. **Payment Service** charges the customer, publishes `PaymentProcessed`.
3. **Shipping Service** attempts to create shipment — fails (e.g. item out of stock). Publishes `ShipmentFailed`.
4. **Payment Service** receives `ShipmentFailed`, runs its **compensating transaction**: issues a refund, publishes `PaymentRefunded`.
5. **Order Service** receives `PaymentRefunded`, runs its **compensating transaction**: updates order status to "CANCELLED".

KEY EXAM DISTINCTION: ORCHESTRATION VS CHOREOGRAPHY

- **Orchestration:** a central *saga orchestrator* service tells each participant what to do next and handles compensation. Easier to trace but adds a central coordinator.
- **Choreography:** each service reacts to events from the previous step with no central controller. More decoupled but harder to trace failures.

The 2023/24 exam specifically asked about **orchestration**.

BUSINESS RISK EXAMPLE (REFER/DEFER EXAM ASKED FOR THIS)

With SAGA, there is a window of time where the order is "PENDING" and payment has been taken but shipping has not yet confirmed. If the system crashes in that window, you may need

reconciliation logic to detect and fix the inconsistency. The business risk is a customer being charged but not receiving goods — acceptable if proper reconciliation and monitoring exist.

13.5 How it appears in past papers

- 2024/25 main asked about connection pooling and read-replica scaling.
- 2022/23 main asked about relational scaling, sharding and consistent hashing.
- 2023/24 asked for the SAGA orchestration pattern.
- Refer/defer tested ACID and distributed transaction difficulty.

QUICK QUIZ — DATA

1. What does referential integrity protect?
2. Why is connection pooling better than opening a fresh connection for every request?
3. What does a read replica improve?
4. What is the core idea of a SAGA?
5. What problem does consistent hashing try to reduce?

▶ Show Answers

EXAM-SAFE SUMMARY LINE

Read replicas help reads, sharding helps distribution, and SAGA helps when one business process spans multiple services and databases.

14. Networking, VNets & CIDR

MEDIUM — older paper but still examinable

This is one of the more technical-looking topics, but the exam usually wants a **sensible network layout** and a **clear explanation of public vs private access**, not deep networking theory.

14.1 Key terms in plain English

Term	Meaning
VNet / virtual network	Your private cloud network space
Subnet	A smaller network carved out inside the larger VNet

CIDR	The notation such as `/16` or `/24` that describes the size of the IP block
Public subnet	Used for components that need direct public access
Private subnet	Used for components that should not be directly reachable from the public inte
Bastion host	A controlled jump box used for admin access into private resources

14.2 CIDR — How It Works From Scratch

An IP address is 32 bits long, written as four groups of numbers (e.g. `10.1.0.0`). CIDR notation adds a `/` suffix that says how many of those 32 bits are the **fixed network part**. The remaining bits are free to use for individual addresses.

CIDR suffix	Fixed bits	Free bits	Number of addresses	Useful for
/16	16	16	65,536	Entire VNet allocation
/18	18	14	16,384	Large AZ block
/24	24	8	256	A single subnet
/28	28	4	16	Very small subnet (e.g. bastion on

> **THE RULE**
>
> Free addresses = $2^{(32 - \text{suffix})}$. So /24 gives 2^8 = 256 addresses. /16 gives 2^{16} = 65,536 addresses. **Smaller suffix = bigger block.**

14.3 Worked CIDR Example — 2022/23 Reassessment Style

The 2022/23 reassessment Q3 gave you a VNet of `10.1.0.0/16` across three availability zones and asked you to:

1. Allocate a CIDR block to each availability zone.
2. Within your chosen AZ, allocate two subnets (one for web/bastion, one for database).

> **EXAMPLE ANSWER WITH WORKING**
>
> **Starting block:** `10.1.0.0/16` — 65,536 addresses total.
>
> **Step 1 — Divide across 3 AZs.** Use /18 per AZ (16,384 addresses each — leaves room to spare):
>
> - AZ-1: `10.1.0.0/18` (10.1.0.0 – 10.1.63.255)

- AZ-2: `10.1.64.0/18` (10.1.64.0 – 10.1.127.255)
- AZ-3: `10.1.128.0/18` (10.1.128.0 – 10.1.191.255)

Step 2 — Subdivide AZ-1 into two subnets. Use /24 (256 addresses each — more than enough for a few servers):

- Public subnet (web server + bastion host): `10.1.0.0/24`
- Private subnet (database server): `10.1.1.0/24`

Rationale: The web server and bastion host need to be reachable from the internet so they go in the public subnet. The database should never be directly reachable from the internet so it goes in the private subnet. A Network Security Group (NSG) on the private subnet only allows traffic from the public subnet's address range.

14.4 Typical Secure Architecture Layout

```
Internet
    |
    ▼
[Load Balancer / Internet Gateway]
    |
    ▼
Public subnet (10.1.0.0/24)
├── Web Server          ← serves HTTP/HTTPS to public users
└── Bastion Host        ← admin SSH entry point only
    |
    ▼ (private traffic only — NSG blocks direct internet access)
Private subnet (10.1.1.0/24)
└── Database Server     ← only reachable from public subnet
```

Bastion host — what it is and why it matters

A **bastion host** (also called a jump box) is a hardened server in the public subnet that is the *only* way to SSH into servers in the private subnet. Administrators connect to the bastion first, then SSH from there into private servers. This way, private servers never need a public IP address.

WHY THIS IS MORE SECURE

The bastion can be heavily locked down — limited to specific admin IP ranges, with strict logging and MFA. If private servers had public IPs directly, each one would be an attack surface.

14.5 Network Peering

Network peering links two separate VNets so resources in each can communicate privately, without going over the public internet. Useful when related systems are in separate VNets but need low-latency private communication.

> **EXAM POINT**
>
> Peering is **private network-to-network connectivity** — not the same as making something publicly accessible.

14.6 How it appears in past papers

- 2022/23 reassessment Q3 — full 25-mark question: design a VNet, allocate CIDR blocks across 3 AZs and 2 subnets, draw the architecture, explain public/admin traffic flow.
- The lecturer's strategy highlights bastion hosts and network peering as topics to know for 2025/26.

> **QUICK QUIZ — NETWORKING & CIDR**
>
> 1. How many addresses does a /24 block contain?
> 2. You have 10.0.0.0/16. What CIDR block would you give to the first /24 subnet?
> 3. Why should a database server be in a private subnet?
> 4. What is a bastion host and why is it used?
> 5. What is network peering?
>
> ▶ Show Answers

15. Infrastructure as Code: Terraform HIGH — recent-paper value

Terraform is lower-frequency than TLS or messaging, but it has appeared with high marks in recent papers, so it is worth knowing the **core vocabulary** and the **declarative vs imperative** distinction.

15.1 Declarative vs imperative

Style	Main idea	Simple memory line
Imperative	You tell the system the steps to perform	"How to do it"
Declarative	You describe the target state you want	"What I want"

15.2 Terraform building blocks

- **Provider:** plugin that talks to the cloud provider API.
- **Resource:** block representing something to create or manage.
- **Variable:** reusable input value.
- **State file:** Terraform's record of what exists.
- **Provisioner:** extra action such as copying files or running commands.
- **null_resource:** lets you run provisioners without claiming a real infrastructure object exists.

> **WHY IAC IS VALUABLE**
>
> It makes infrastructure **repeatable**, **versionable**, **reviewable** and easier to rebuild than clicking around manually in a portal.

15.3 Provisioners in plain English

- **file provisioner:** copies a file from your local machine to the remote target machine.
- **remote-exec:** runs a list of shell commands on the remote machine over SSH.
- **local-exec:** runs a command on the machine where Terraform itself is running (your laptop or CI server).

15.4 Annotated code example — 2024/25 main exam style

The 2024/25 main Q2 gave a `null_resource` block and asked you to explain it by line number (20 of 25 marks). This is exactly the kind of thing you need to be able to do.

```
# Line 1
resource "null_resource" "deploy_app" {

  # Lines 2–4: triggers block
  triggers = {
    always_run = timestamp()
  }

  # Lines 6–16: file provisioner
  provisioner "file" {
    source      = "./app.zip"
    destination = "/home/ubuntu/app.zip"

    connection {
      type        = "ssh"
      user        = "ubuntu"
      private_key = file("~/.ssh/id_rsa")
      host        = aws_instance.web.public_ip
```

```
    }
  }

  # Lines 17–27: remote-exec provisioner
  provisioner "remote-exec" {
    inline = [
      "cd /home/ubuntu",
      "unzip -o app.zip",
      "npm install",
      "pm2 restart app"
    ]
    connection {
      type        = "ssh"
      user        = "ubuntu"
      private_key = file("~/.ssh/id_rsa")
      host        = aws_instance.web.public_ip
    }
  }
}
```

HOW TO ANSWER EACH LINE-REFERENCE QUESTION

Line 1 — `null_resource "deploy_app"` : A `null_resource` is a Terraform resource that represents no real infrastructure. Its purpose is to act as a hook for running provisioners — actions like copying files or running commands — that are not tied to creating a specific cloud resource. It is used here to run the deployment steps against an already-existing VM.

Lines 2–4 — `triggers` : The `triggers` block controls when Terraform will re-run this resource. `always_run = timestamp()` means the timestamp changes on every Terraform apply, which forces the `null_resource` to be treated as changed every time. This ensures the provisioners re-run on every deployment rather than being skipped because Terraform thinks nothing has changed.

Lines 6–16 — `file` provisioner: The file provisioner copies a local file (`./app.zip`) to a specific path on the remote machine (`/home/ubuntu/app.zip`) using SSH. The `connection` block inside it specifies how to SSH into the machine — using the ubuntu user, a private key file, and the VM's public IP address.

Lines 17–27 — `remote-exec` provisioner: The remote-exec provisioner runs a list of shell commands on the remote machine over SSH after the file has been copied. Here it navigates to the home directory, unzips the uploaded archive, installs Node.js dependencies with npm, and restarts the application process using pm2. This completes the deployment without any manual SSH session.

15.5 How it appears in past papers

- 2024/25 main Q2 — 20 of 25 marks on explaining a Terraform code block by line number (`null_resource` , triggers, file provisioner, remote-exec provisioner).
- Same paper also asked: why is declarative IaC preferred over imperative scripting? (5 marks).

DECLARATIVE VS IMPERATIVE — 5-MARK MODEL ANSWER

Declarative IaC such as Terraform lets you describe the *desired end state* of your infrastructure rather than writing step-by-step scripts. Terraform works out the correct order of operations and only makes the changes needed to reach the target state. This makes infrastructure more repeatable, versionable in source control, and easier to rebuild from scratch. Imperative approaches such as bash scripts require you to handle ordering and error recovery manually, which becomes fragile at scale.

QUICK QUIZ — TERRAFORM

1. What is Infrastructure as Code?
2. Why is declarative IaC often preferred over imperative scripting?
3. What is the job of Terraform's state file?
4. What does a file provisioner do?
5. Why might a `null_resource` be useful?

▶ Show Answers

HOW TO ANSWER LINE-REFERENCE QUESTIONS

Do not just repeat the syntax. Explain the **purpose** of the block in the wider deployment flow.

16. Code-Reading Practice MEDIUM — useful exam skill

Some papers give code or config and ask what it does. The fastest way to improve here is to stop reading character-by-character and instead identify the **role** of each block.

16.1 A four-step method

1. **Identify the language or file type**: Express JS, Dockerfile, compose YAML, Terraform,

Kubernetes YAML.

2. **Find the high-level purpose**: routing, deployment, networking, persistence, token handling.
3. **Look for the key nouns**: route, middleware, image, volume, Deployment, ReplicaSet, provider, resource.
4. **Translate the block into plain English** before writing your answer.

16.2 What to look for in common file types

File type	What to look for first
Express code	Route method, path, middleware order, response codes
Dockerfile	Base image, copied files, exposed port, startup command
Compose YAML	Services, ports, environment variables, volumes, networks
Kubernetes YAML	Kind, replicas, image, service type, selectors
Terraform	Provider, resource blocks, variables, provisioners, null_resource

16.3 How to write the answer

USEFUL EXAM WORDING

- "This block defines..."
- "The purpose of this line is..."
- "This would cause the application to..."
- "In the wider architecture, this allows..."

COMMON MISTAKE

Students often describe syntax but never explain the operational meaning. Marks are usually for **purpose**, not for reading symbols aloud.

QUICK QUIZ — CODE READING

1. What should you identify before reading individual lines in detail?
2. What is usually the first thing to find in an Express snippet?

3. What does a volume line in Compose usually suggest?
4. Why are selectors important in Kubernetes YAML?
5. What do examiners normally want more than raw syntax description?

▶ Show Answers

17. Key Diagrams to Memorise HIGH — repeated mark source

This module repeatedly rewards simple labelled diagrams. They do not need to be pretty. They do need to be **clear, labelled and referred to in your written explanation**.

17.1 High-value diagrams

- Client -> API Gateway -> multiple microservices.
- Client -> Load Balancer -> multiple app instances across two availability zones.
- Deployment -> old ReplicaSet and new ReplicaSet -> pods during a rolling update.
- Browser -> Identity Provider -> code -> token exchange -> protected app resource.
- Browser -> certificate/handshake -> session keys -> encrypted traffic.
- Producer -> queue or exchange -> one consumer or many subscribers.
- Primary database -> read replicas -> read traffic spread out.

17.2 Tiny diagram templates

```
API Gateway
Client -> Gateway -> Service A
                  -> Service B
                  -> Service C

Load Balancer
Users -> LB -> App 1 (AZ1)
            -> App 2 (AZ2)

Rolling update
Deployment -> Old ReplicaSet -> old pods
           -> New ReplicaSet -> new pods
```

17.3 What labels you must not forget

LABEL CHECKLIST

- Direction arrows
- Role names such as browser, gateway, IdP, queue, load balancer, ReplicaSet
- Where the security or trust check happens
- Where traffic is distributed or where events flow
- Which step is synchronous and which is asynchronous where relevant

QUICK QUIZ — DIAGRAMS

1. Why can a simple diagram earn marks in this module?
2. What is the most common problem with weak exam diagrams?
3. What should arrows show?
4. Which diagram is especially useful for messaging questions?
5. Why should you refer to the diagram in your written answer?

▶ Show Answers

18. Exam Technique

CRITICAL — converts knowledge into marks

This exam is not just about knowing terms. It is about recognising the **command word**, giving the **right depth for the marks**, and using **structure** so the examiner can award marks quickly.

18.1 Command words

Command word	What the examiner wants	Best structure
Explain	Definition + process + why it matters	Define, then walk through steps in ord
Compare	Both sides against the same criteria	Same headings for both, then conclud
Discuss	Balanced strengths and weaknesses	Pros, cons, context, judgement
Evaluate	Judge against criteria	State criteria, weigh evidence, conclu
Justify	Choose and defend	Choice first, then reasons linked to sc
Design	Propose something sensible and explain it	Show design, label it, justify each part

18.2 Mark-depth guide

Marks	Usually enough
5	Definition + 2 or 3 explained points + one small example
10	Short but developed explanation, often with a process or one balanced comparison
15	Structured answer with several developed points and often a diagram
25	Full answer with process, examples, trade-offs and a conclusion

18.3 Diagram questions

RELIABLE METHOD

1. Draw the simple labelled diagram early.
2. Number the stages if the process has order.
3. Use the diagram in your explanation: "At step 3..."
4. Do not rely on the diagram alone — explain what it shows.

18.4 Common ways marks are lost

- Giving a definition only when the question wanted a process.
- Ignoring the use-case in the question.
- Listing only advantages and no trade-offs.
- Mixing up close terms such as authentication/authorisation or queue/pub-sub.
- Writing too much on a low-mark sub-part and running out of time later.

18.5 Before moving on checklist

FINAL 15-SECOND CHECK

- Did I answer the exact command word?
- Did I define the key term?
- Did I explain the mechanism or comparison clearly?
- Did I use the scenario in the question?
- Did I include a limitation, trade-off or judgement where needed?
- Would a simple diagram help?

QUICK QUIZ — EXAM TECHNIQUE

1. What should usually come right after the definition in an "explain" answer?
2. What is the biggest weakness of writing two disconnected mini-essays in a compare question?
3. What must a good evaluate answer include near the end?
4. Why should you draw a diagram early if the question invites one?
5. What is one common reason students drop marks even when they know the topic?

▶ Show Answers

19. Rapid-Fire Review CRITICAL — final pass section

Use this section in the last few days and again the day before the exam. It is for **fast recall**, not first-time learning.

19.1 Highest-value topics

- TLS, certificates, digital signatures and HTTPS process questions.
- Messaging patterns: synchronous vs asynchronous, queues, pub-sub and event-driven flow.
- Monolith vs microservices, including trade-offs and API gateway reasoning.
- API design, Express middleware, CORS and OpenAPI documentation.
- Cloud models, resilience, load balancing and availability zones.
- Containers, named volumes, environment variables, Kubernetes objects and rolling updates.
- Identity and access: authentication, authorisation, OAuth, OIDC, JWT and SSO.

19.2 Quick-fire definitions

Term	Fast definition
Loose coupling	Minimal dependence on another component's internal details.
Middleware	Code that runs between receiving a request and sending a response.
CORS	Browser rule controlling whether one origin may call another.
Named volume	Docker-managed persistent storage mounted into a container.
Digital certificate	Signed document binding a public key to an identity.

JWT	Compact token commonly structured as header.payload.signature.
Read replica	Read-only copy used to spread read traffic away from the primary DB.
SAGA	Distributed transaction pattern using local steps plus compensation.

19.3 Common compare pairs

Pair	Core contrast
Monolith vs microservices	Simplicity vs agility and independent scaling
HTTP vs HTTPS	Plain transport vs transport protected by TLS
Queue vs pub-sub	One consumer vs many subscribers
VM vs container	Full OS isolation vs lighter shared-kernel isolation
IaaS vs PaaS	More control/responsibility vs less platform work
Thick vs thin event	Carry state vs carry minimal notification data
NodePort vs ClusterIP vs LoadBalancer	Node exposure vs internal only vs cloud-integrated exposure

19.4 Sequences to memorise

- **TLS:** ClientHello -> ServerHello + certificate -> certificate validation -> key agreement -> session keys -> encrypted traffic.
- **Digital signature verification:** hash message -> sign with private key -> verify with public key -> compare digests.
- **OIDC flow:** protected page -> redirect to IdP -> login -> code returned -> token exchange -> session created.
- **Kubernetes rolling update:** new ReplicaSet up -> new pods healthy -> traffic shifts -> old ReplicaSet down.
- **Event-carried state transfer:** source data change -> publish event with state -> subscribers update local copies -> eventual consistency.

> **HOW TO USE THIS SECTION**
>
> Cover one column or one bullet line, try to say it from memory, then uncover and check yourself. This is much more effective than just staring at the page.